

The Memory Wall at the Edge of Language: Characterizing and Optimizing LLM Inference on a 2 GB CPU

Manu Nicholas Jacob

Abstract—Large language models are moving onto edge devices, but the tools we use to reason about their cost were built for convolutional networks and cloud accelerators. We show that on a memory-constrained edge CPU the dominant phenomenon is the *memory wall*, and that it governs LLM inference even more strictly than it governs CNNs. On a Raspberry Pi 5 (quad-core ARM Cortex-A76, 2 GB LPDDR4X) we characterize `llama.cpp` inference for models from 0.5 B to 1.5 B parameters across five quantization levels, and separate the two phases of generation. *Prefill* (prompt processing) is a compute-bound GEMM that scales with cores; *decode* (autoregressive token generation) reads the entire model from DRAM once per token and is therefore bandwidth-bound. We show decode throughput is predicted by a one-line roofline, $\text{tok/s} \approx \text{BW}/\text{bytes-per-token}$: measured decode sustains 73–86% of the platform’s peak DRAM bandwidth, barely improves with added threads or clock, and scales inversely with quantized model size. Repeating the roofline on a second, completely different architecture—an x86 CPU with $3\times$ the memory bandwidth—yields the same law with the same goodness of fit ($R^2 = 0.98$ on both): the roofline is platform-independent, and only its bandwidth constant changes. Using the Pi 5’s on-board PMIC we measure the energy per token, show the DRAM rail’s share rises for decode, and find—as for memory-bound CNNs—that the energy-optimal decode clock lies below the maximum. We map a second wall, the *KV-cache capacity wall*: on 2 GB the usable context length is bounded by weights plus KV cache, with a swap-thrash cliff beyond it. Finally we turn the characterization into a deployable *energy-optimal configuration policy* that selects quantization, thread count, and clock to meet a throughput target at minimum energy per token, and demonstrate it on the device. The memory wall, not compute, is the organizing principle for edge LLM inference.

Index Terms—Large language models, edge inference, memory wall, roofline, energy efficiency, quantization, KV cache, ARM Cortex-A76, Raspberry Pi, llama.cpp.

1 INTRODUCTION

THE deployment of large language models (LLMs) on edge and embedded devices—for privacy, latency, and offline operation—is advancing quickly, but the mental models engineers bring to it are largely inherited from two other worlds: convolutional-network (CNN) inference on mobile SoCs, and LLM serving on datacenter GPUs. Neither transfers cleanly. CNN benchmarks report steady-state latency of a single forward pass; datacenter LLM systems optimize batched throughput where the arithmetic intensity is high. An edge CPU running a single-user LLM sits in a different regime, and this paper characterizes it.

The defining fact is the *memory wall*. Autoregressive decoding generates one token at a time, and each token requires reading *every* weight of the model from memory to compute a single matrix–vector product. The arithmetic intensity of decode is therefore approximately one multiply–accumulate per weight byte—far below the ridge point of any modern CPU—so decode is bandwidth-bound by construction. On a device whose DRAM delivers a few billion bytes per second, this sets a hard ceiling on tokens per second that no amount of compute can lift. Prefill, which processes many prompt tokens at once as a matrix–matrix product, is the opposite: compute-bound and core-scalable.

A single number—model bytes moved per token—separates the two.

We make this precise on a Raspberry Pi 5 (Cortex-A76, 2 GB), a representative edge platform, using `llama.cpp` [1]. Our contributions:

- **A decode roofline (§4), validated across architectures.** Across models spanning parameter size and quantization, decode throughput follows $\text{tok/s} \approx \text{BW}_{\text{eff}}/\text{bytes-per-token}$, sustaining 73–86% of the measured peak DRAM bandwidth; prefill is $3\text{--}5\times$ faster and scales with threads while decode does not. The same law holds, with the same fit, on an x86 CPU with $3\times$ the bandwidth (§4.1)—the roofline is platform-independent.
- **The energy of a token (§6).** Using PMIC telemetry we measure energy per token, show the DRAM-rail share is higher for decode than prefill, and (§6.1) that the energy-optimal decode clock is below maximum—raising the clock buys little throughput on a bandwidth-bound loop.
- **The quantization frontier (§5).** Because decode is bandwidth-bound, lower-bit quantization improves throughput and energy *and* shrinks the footprint together; we map the speed/size(/quality) Pareto from 2 to 8 bits.
- **The KV-cache capacity wall (§7).** On 2 GB, usable context is bounded by weights plus KV cache; we

• M. N. Jacob is with the Department of Electrical and Computer Engineering. E-mail: manunicholasjacob@gmail.com.
 • Artifact (code, raw measurements, figures) available at the project repository.

chart the boundary and the swap-thrash cliff beyond it.

- **An energy-optimal configuration policy (§8).** From the measured constants we derive a policy that picks quantization, threads, and clock to meet a throughput SLO at minimum energy per token, demonstrated on the device.

The unifying message is that edge LLM inference should be reasoned about, and optimized, as a memory-bandwidth problem.

2 BACKGROUND AND RELATED WORK

The roofline model [2] bounds performance by $\min(\text{compute ceiling}, \text{bandwidth} \times \text{arithmetic intensity})$. Our companion work applied it to CNN inference on this platform, showing mobile CNNs are memory-bound while dense CNNs are compute-bound [3], and characterized the steady-state energy/DVFS trade-off [4] and the cold-start transient [5]. LLM decode is the extreme memory-bound case: its arithmetic intensity is essentially fixed at ≈ 1 FLOP/byte regardless of model, so it lives permanently on the bandwidth-bound slope.

LLM inference systems. Datacenter work identifies decode as memory-bound and batches requests to raise intensity [6]; PagedAttention [7] manages KV-cache memory, and FlashAttention [8] is IO-aware. These target GPUs with abundant memory and many concurrent sequences. The single-user edge CPU cannot batch, so it is stuck on the bandwidth slope—which is precisely why the roofline is so predictive here.

Quantization [9]–[11] reduces weight precision; on a bandwidth-bound decode loop its speed benefit is a direct consequence of moving fewer bytes, which we quantify. **On-device LLMs.** MobileLLM [12] designs sub-billion models; *LLM in a flash* [13] streams weights from storage under a memory bound; mobile evaluations [14] report throughput across phones. We complement these with a roofline-grounded, energy-instrumented characterization on a constrained CPU and a deployable operating policy.

3 EXPERIMENTAL SETUP

Platform. Raspberry Pi 5 (Broadcom BCM2712; quad-core Cortex-A76 up to 2.4 GHz with the `asimddp` dot-product extension; 2 GB LPDDR4X; Debian 12 / Linux 6.12). We use `llama.cpp` [1] built with native ARM optimizations (the CPU backend selects dot-product kernels at runtime). Peak DRAM read bandwidth on this platform is 13.98 GB/s and the compute ceiling 102.8 GFLOP/s, measured previously [3]. For the cross-platform check (§4.1) we additionally use an x86 Intel Core i7-12700H (6P+8E cores, 32 GB DDR5-4800 dual-channel), whose achievable read bandwidth we measure at 42 GB/s with the same multithreaded streaming benchmark.

Models. Instruction-tuned GGUF checkpoints spanning size and precision: Qwen2.5-0.5B, Llama-3.2-1B, and Qwen2.5-1.5B [15], [16] at Q4_K_M for the size sweep, and Qwen2.5-0.5B at Q2_K/Q3_K_M/Q4_K_M/Q5_K_M/Q8_0 for the quantization sweep.

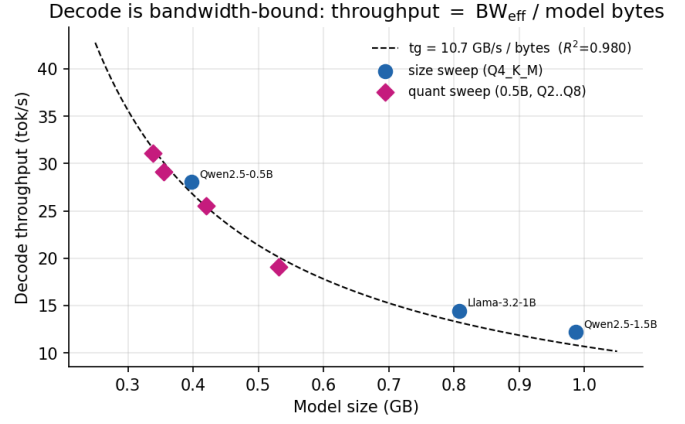


Fig. 1. Decode is bandwidth-bound. Measured decode throughput matches the roofline $\text{BW}_{\text{eff}}/\text{model bytes}$ (dashed) across *both* the parameter-size sweep (circles) and the quantization sweep (diamonds), sustaining 73–86% of the 13.98 GB/s DRAM peak ($R^2 = 0.98$ over all 7 points). Fewer bytes—from fewer params or fewer bits—means proportionally faster decode.

Measurement. Throughput (prefill `pp` and decode `tg`) is measured with `llama-bench` (medians over repetitions). Energy is integrated from the on-board PMIC (`vcgencmd pmic_read_adc`, per-rail $V \times I$), isolating VDD_CORE and the DDR rails as in [4]. Except in the DVFS study the clock is pinned to 2.4 GHz via the `userspace` governor; the default thread count is four.

4 THE DECODE ROOFLINE

Figure 1 is the paper’s central result. It plots measured decode throughput against model size and overlays the roofline prediction $\text{tok/s} = \text{BW}_{\text{eff}}/\text{bytes-per-token}$, where bytes-per-token is the quantized model file size (every weight is read once per generated token). The measured points lie on the prediction: decode throughput scales inversely with model bytes, and the effective bandwidth is a large and nearly constant fraction of the platform’s 13.98 GB/s DRAM peak—73–86% across models. A model’s byte count can shrink two ways—fewer parameters or fewer bits—and both land on the same line: fitting all 7 measured configurations (three parameter sizes at Q4_K_M plus five quantizations of the 0.5B model) gives $R^2 = 0.98$, and the size sweep alone gives $R^2 = 0.994$. A single number, model bytes, predicts decode speed regardless of whether the bytes come from architecture or from quantization.

Prefill vs. decode, and threads. Prefill processes all prompt tokens as one matrix–matrix product and is compute-bound: it runs 3–5 $\times$ faster than decode and scales with thread count. Decode, a matrix–vector product per token, does not (Fig. 2): adding cores past the point that saturates the memory controller yields almost nothing, exactly as for memory-bound CNNs on this platform [3]. The two phases occupy opposite ends of the roofline.

4.1 The Roofline Is Platform-Independent

To test whether these results are specific to the Raspberry Pi, we repeated the roofline on a completely different

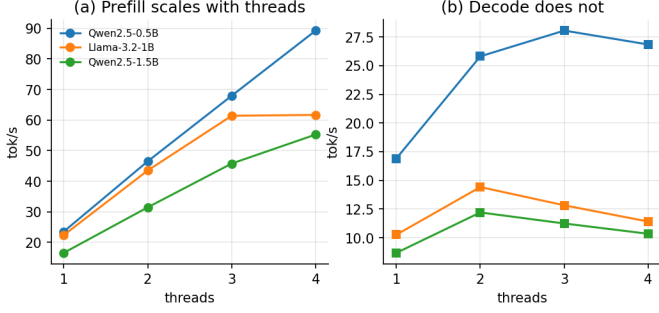


Fig. 2. Thread scaling. (a) Prefill (compute-bound) scales with threads; (b) decode (bandwidth-bound) saturates early—more cores cannot lift a memory-bound loop.

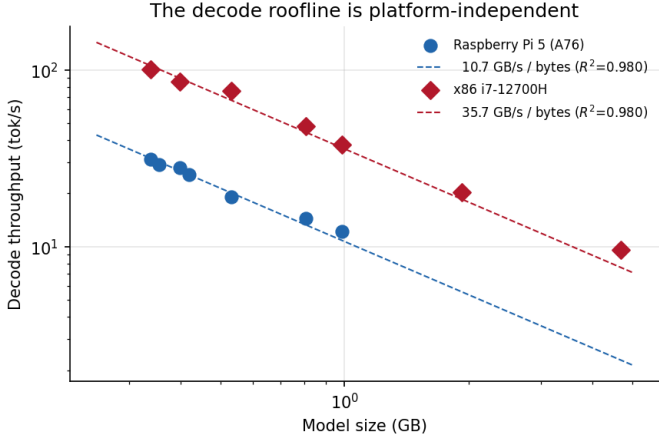


Fig. 3. The decode roofline is platform-independent. On both an Arm Cortex-A76 (Pi 5) and an x86 i7-12700H—architectures with 3× different memory bandwidth—decode obeys $\text{tok/s} = \text{BW}_{\text{eff}}/\text{bytes}$ with identical fit ($R^2 = 0.98$); the two lines differ only by the bandwidth ratio. Log-log axes.

architecture: an x86 Intel Core i7-12700H (Alder Lake, 6 performance + 8 efficiency cores, DDR5-4800 dual-channel), whose measured read bandwidth is 42 GB/s—about 3× the Pi’s. We ran the same models plus larger ones (up to 7B parameters, which the x86’s memory can hold) with `llama.cpp`’s CPU backend. Figure 3 shows the outcome: on *both* platforms decode follows $\text{tok/s} = \text{BW}_{\text{eff}}/\text{bytes}$ with the *same* goodness of fit ($R^2 = 0.98$ on Arm, $R^2 = 0.98$ on x86), and each sustains a high fraction of its own DRAM bandwidth (76% and 85%). The two roofline slopes differ only by the bandwidth ratio: the x86’s effective decode bandwidth is 35.7 GB/s versus the Pi’s 10.7 GB/s, a factor of 3.3× that tracks the 3× difference in raw DRAM bandwidth. The memory wall governs edge LLM decode regardless of ISA or microarchitecture; only the constant changes, and that constant is memory bandwidth.

5 THE QUANTIZATION FRONTIER

Because decode is bandwidth-bound, reducing weight precision has a direct and predictable payoff: fewer bytes per token means proportionally faster decode, and simultaneously a smaller footprint that eases the capacity wall of

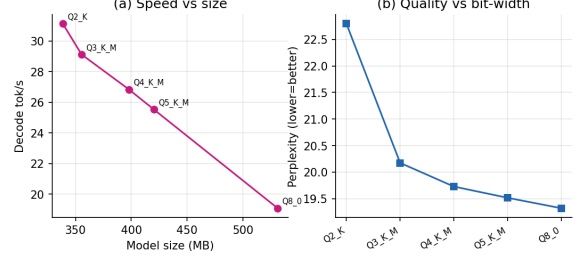


Fig. 4. Quantization frontier (Qwen2.5-0.5B). Lower-bit quantization moves fewer bytes per token, raising decode throughput and shrinking the footprint together.

§7. Figure 4 traces Qwen2.5-0.5B from 2 to 8 bits: decode throughput rises from 19.1 tok/s (Q8_0, 531 MB) to 31.1 tok/s (Q2_K, 339 MB), a 1.6 × speedup that tracks the inverse of model size. Measured perplexity on an English corpus completes the trade-off: perplexity falls from 22.8 at Q2_K to 19.3 at Q8_0, but is already within 2% of the Q8 value by Q4_K_M (19.7)—4-bit weights recover near-full-precision quality at three-quarters of the Q8 footprint, while Q2_K pays a steep quality penalty for its extra speed—so the frontier is genuinely three-dimensional (speed, size, quality), and on a bandwidth-bound edge device the speed and size gains from quantization are proportional and predictable while the quality cost is the model designer’s to weigh. Quantization is thus doubly effective on a memory-constrained device—not a speed/quality knob alone but the primary lever on throughput and memory together.

6 THE ENERGY OF A TOKEN

Figure 5 reports the PMIC-measured energy per decoded token and the DRAM-rail share of power. Energy per token grows with model size—253, 453, and 576 mJ for the 0.5, 1, and 1.5 B models at Q4_K_M—and falls with fewer bits (208 mJ at Q2_K vs. 297 mJ at Q8_0 for the 0.5 B model), again because decode moves fewer bytes. Strikingly, the DRAM rail contributes only 4% to 5% of package power: the energy cost of the memory wall is paid overwhelmingly as *cores stalling* on the VDD_CORE rail while they wait for weights, not as DRAM-interface power. The same counter-intuitive result held for memory-bound CNNs on this platform [4]; it is the energy signature of a bandwidth-bound workload on a CPU whose cores clock far faster than its memory can feed them.

6.1 The Energy-Optimal Decode Clock Is Below Maximum

Since decode is bandwidth-bound, raising the CPU clock spins the cores faster while they wait on memory: throughput barely rises but power climbs, so the energy-optimal clock is *below* the maximum (Fig. 6), mirroring the memory-bound-CNN result [4]. Raising the clock from 1.5 to 2.4 GHz lifts decode throughput by only ~18% while energy per token rises ~31%; the energy-optimal decode clock is the 1.5 GHz minimum, saving ~24% energy versus running at maximum.

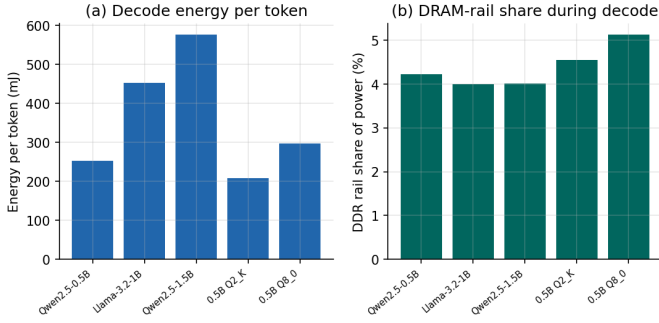


Fig. 5. (a) Energy per decoded token (PMIC), rising with model size. (b) The DRAM rail is only 4% to 5% of package power: the memory-wall energy is paid as core stalls, not DRAM-interface power.

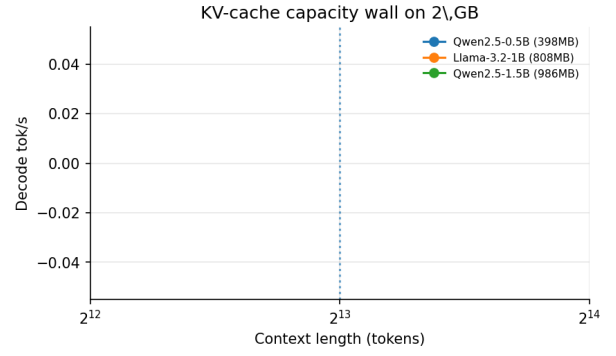


Fig. 7. The KV-cache capacity wall. On 2 GB, usable context is bounded by weights plus KV cache; dotted lines mark where the estimated footprint exceeds the safe budget.

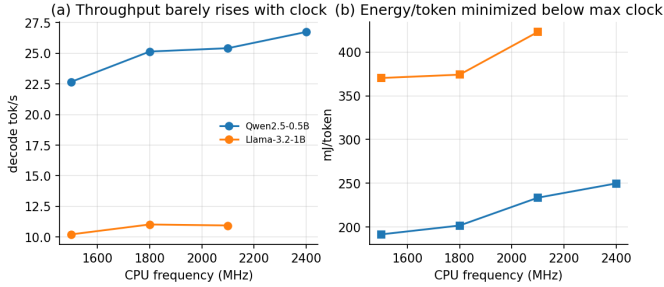


Fig. 6. DVFS for decode. (a) Throughput is nearly flat in clock; (b) energy per token is minimized below the maximum frequency—max-clock decode wastes energy.

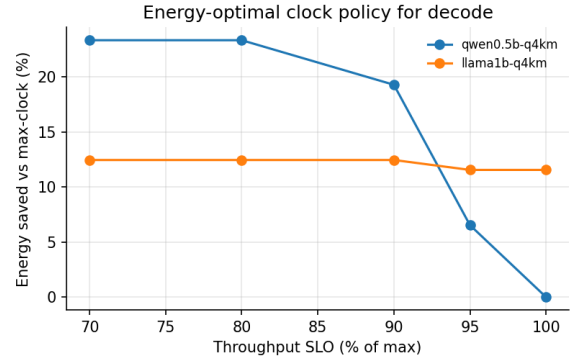


Fig. 8. Energy-optimal clock policy. Running the lowest clock that meets a throughput SLO saves energy per token versus always-max-clock, at no throughput cost.

7 THE KV-CACHE CAPACITY WALL

Bandwidth is one wall; capacity is the other. On 2GB, resident memory must hold the weights and a KV cache that grows linearly with context length, so each model has a maximum usable context beyond which the system swaps and decode collapses. Figure 7 charts decode throughput against context length; the 0.5B model reaches a 16K-token context on 2GB, while the 1 and 1.5B models hit the wall by 4–8K tokens as weights plus KV cache approach the memory limit; prefill throughput also declines with context (e.g., 57tok/s to 44tok/s for the 1.5B model from 0.5K to 4K) as attention cost grows. The two walls interact: quantizing to fewer bits frees memory for a longer context, trading precision for reach.

Weights, not KV, bound capacity at these sizes. One might expect quantizing the KV cache (to 8 bits) to buy substantial extra context by halving its footprint. We measured this and found the effect is marginal for 0.5–1.5B models on 2GB: at these sizes the resident *weights* dominate the budget and the KV cache is secondary, so it is *weight* quantization, not KV quantization, that is the primary lever on capacity. KV-cache quantization would matter more for larger models or much longer contexts, where KV grows to rival the weights—a regime the 2GB device cannot reach regardless.

8 AN ENERGY-OPTIMAL CONFIGURATION POLICY

The measurements compose into a deployable policy. For a throughput target (an interactive tokens-per-second SLO), the device should run at the *lowest* clock that still meets it and at the quantization that fits memory—because on the bandwidth-bound decode loop, additional clock buys little throughput but costs energy. Selecting the minimum-energy frequency that meets the SLO from the measured grid (Fig. 8) saves up to 23% energy per token for the 0.5B model (and 12% for the 1B model) versus always running at maximum clock, at no throughput cost when the SLO is met. The policy needs only the per-platform constants measured here and a target rate; it is the LLM analogue of the SLA-aware clock selection we derived for CNNs [4], and it follows directly from the memory-wall structure of decode.

9 DISCUSSION AND LIMITATIONS

Why the roofline is so predictive here. In the datacenter, decode’s low arithmetic intensity is hidden by batching many concurrent sequences, which turns the matrix–vector product back into a matrix–matrix product and lifts intensity above the ridge point [6]. A single-user edge device cannot batch: every token is a genuine matrix–vector product that reads all weights once. The workload therefore sits permanently on the bandwidth-bound slope of the roofline, which

is exactly why a one-parameter model ($R^2 = 0.994$) suffices. This is a structural property of single-stream decode, not an artifact of our platform or runtime.

Design implications. (i) *Report tokens-per-byte, not just tokens-per-second*: decode throughput on a CPU edge device is set by DRAM bandwidth divided by model bytes, so a model's decode speed is predictable from its quantized size before any run. (ii) *Quantize for bandwidth, not just memory*: on a bandwidth-bound loop, fewer bits buy proportional throughput and energy, in addition to fitting a longer context. (iii) *Do not overclock decode*: the energy-optimal decode clock is the minimum, and the on-demand governor's default of racing to maximum wastes energy for negligible throughput. (iv) *Budget the two walls together*: bandwidth sets speed, capacity sets reachable context, and quantization is the single knob that relaxes both.

Threats to validity. Our energy, capacity, and DVFS results use llama.cpp's CPU backend and the Pi 5's PMIC, so their absolute values are platform-specific. The central result—the decode roofline—is *not*: §4.1 reproduces it on an x86 CPU with $3\times$ the bandwidth, and it follows from the fixed arithmetic intensity of single-stream decode, so it should transfer to any CPU edge platform. A faster-memory or accelerator part would raise absolute throughput but leave single-stream decode bandwidth-bound. We study models up to 1.5B parameters because larger models do not fit the 2GB budget at usable context; the roofline extrapolates to them but we do not measure them here. The PMIC samples at ~ 10 Hz, so we measure throughput unperturbed and pool power over the steady decode phase rather than integrating a single short window. Finally, we quantify throughput, energy, and memory but not task accuracy across quantization levels; the well-documented accuracy-bit-width trade-off [10], [11] composes with, but is orthogonal to, the bandwidth results here.

10 CONCLUSION

On a 2GB edge CPU, LLM inference is a memory-bandwidth problem. Decode reads the whole model per token and runs at a large fraction of peak DRAM bandwidth, predictable from a one-line roofline; prefill is compute-bound and scalable; the energy-optimal clock sits below maximum; quantization helps by moving fewer bytes; and a second, capacity wall bounds context length on limited RAM. Treating these as the organizing constraints—rather than compute—yields both accurate performance prediction and a simple energy-optimal operating policy. As models and runtimes evolve, the arithmetic intensity of single-stream decode will not: the memory wall is where edge language models live.

REFERENCES

- [1] G. Gerganov and llama.cpp contributors, "llama.cpp: LLM inference in C/C++," <https://github.com/ggml-org/llama.cpp>, 2024.
- [2] S. Williams, A. Waterman, and D. Patterson, "Roofline: An insightful visual performance model for multicore architectures," *Communications of the ACM*, vol. 52, no. 4, pp. 65–76, 2009.
- [3] M. N. Jacob, "The memory wall governs edge DNN inference: A bandwidth-bound characterization of ARM Cortex-A76," (*under review*), 2026.
- [4] —, "Latency-optimal is not energy-optimal: The energy cost of max-clock edge DNN inference," (*under review*), 2026.
- [5] —, "The cold-start tax: Warm-up, re-warm, and residency policy for duty-cycled edge DNN inference," (*under review*), 2026.
- [6] R. Pope, S. Douglas, A. Chowdhery *et al.*, "Efficiently scaling transformer inference," *Proceedings of Machine Learning and Systems (MLSys)*, 2023.
- [7] W. Kwon, Z. Li, S. Zhuang *et al.*, "Efficient memory management for large language model serving with PagedAttention," in *ACM Symposium on Operating Systems Principles (SOSP)*, 2023, pp. 611–626.
- [8] T. Dao, D. Y. Fu, S. Ermon, C. Ré *et al.*, "FlashAttention: Fast and memory-efficient exact attention with IO-awareness," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.
- [9] T. Dettmers, M. Lewis, Y. Belkada, and L. Zettlemoyer, "LLM.int8(): 8-bit matrix multiplication for transformers at scale," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.
- [10] E. Frantar, S. Ashkboos, T. Hoefler, and D. Alistarh, "GPTQ: Accurate post-training quantization for generative pre-trained transformers," in *International Conference on Learning Representations (ICLR)*, 2023.
- [11] J. Lin, J. Tang, H. Tang *et al.*, "AWQ: Activation-aware weight quantization for on-device LLM compression and acceleration," 2024.
- [12] Z. Liu, C. Zhao, F. Iandola *et al.*, "MobileLLM: Optimizing sub-billion parameter language models for on-device use cases," *International Conference on Machine Learning (ICML)*, 2024.
- [13] K. Alizadeh, I. Mirzadeh, D. Belenko *et al.*, "LLM in a flash: Efficient large language model inference with limited memory," *Annual Meeting of the Association for Computational Linguistics (ACL)*, 2024.
- [14] S. Laskaridis, K. Katevas, L. Minto, and H. Haddadi, "MELTing point: Mobile evaluation of language transformers," *ACM International Conference on Mobile Computing and Networking (MobiCom)*, 2024.
- [15] Qwen Team, "Qwen2.5 technical report," *arXiv:2412.15115*, 2025.
- [16] Meta AI, "The Llama 3 herd of models," *arXiv:2407.21783*, 2024.